

Efficient SAR Vessel Detection for FPGA-Based On-Satellite Sensing

Colin Laganier^{*}
The Alan Turing Institute
London, United Kingdom

Liam Fletcher
The Alan Turing Institute
London, United Kingdom

Elim Kwan
The Alan Turing Institute
London, United Kingdom

Richard Walters
The Alan Turing Institute
London, United Kingdom

Victoria Nockles
The Alan Turing Institute
London, United Kingdom

Abstract

Rapid analysis of satellite imagery within minutes-to-hours of acquisition is increasingly vital for many remote sensing applications, and is an essential component for developing next-generation autonomous and distributed satellite systems. On-satellite machine learning (ML) has the potential for such rapid analysis, by overcoming latency associated with intermittent satellite connectivity to ground stations or relay satellites, but state-of-the-art models are often too large or power-hungry for on-board deployment. Vessel detection using Synthetic Aperture Radar (SAR) is a critical time-sensitive application in maritime security that exemplifies this challenge. SAR vessel detection has previously been demonstrated only by ML models that either are too large for satellite deployment, have not been developed for sufficiently low-power hardware, or have only been tested on small SAR datasets that do not sufficiently represent the difficulty of the real-world task. Here we systematically explore a suite of architectural adaptations to develop a novel YOLOv8 architecture optimized for this task and FPGA-based processing. We deploy our model on a Kria KV260 MPSoC, and show it can analyze a ~700 megapixel SAR image in less than a minute, within common satellite power constraints (<10W). Our model has detection and classification performance only ~2% and 3% lower than values from state-of-the-art GPU-based models on the largest and most diverse open SAR vessel dataset, xView3-SAR, despite being ~50 and ~2500 times more computationally efficient. This work represents

a key contribution towards on-satellite ML for time-critical SAR analysis, and more autonomous, scalable satellites.¹

CCS Concepts

• **Computing methodologies** → **Computer vision**; • **Hardware** → **Reconfigurable logic and FPGAs**; • **Applied computing** → **Earth and atmospheric sciences**.

Keywords

Edge AI, FPGA, Object Detection, On-Satellite Machine Learning, Synthetic Aperture Radar

ACM Reference Format:

Colin Laganier, Liam Fletcher, Elim Kwan, Richard Walters, and Victoria Nockles. 2025. Efficient SAR Vessel Detection for FPGA-Based On-Satellite Sensing. In *The Tenth ACM/IEEE Symposium on Edge Computing (SEC '25)*, December 3–6, 2025, Arlington, VA, USA. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/3769102.3772713>

1 Introduction

Rapid analysis of low earth orbit satellite imagery within hours of data acquisition is vital for a diverse range of remote sensing applications, from wildfire detection, to agricultural monitoring, to disaster response [6, 8, 65], and latency requirements for many applications are moving beyond near real-time (1-3 hours), to quasi real-time (<30 mins), with target latencies of <5 minutes in some cases [39]. Furthermore, these shortest latency requirements will be increasingly important due to growing interest in autonomous and distributed satellite capabilities [4], for example “tip-and-cue” processes [15] (where one satellite uses initial analysis to request further imagery from another satellite). Such autonomous tasking and inter-satellite coordination capabilities have been identified as critical for next-generation Earth observation systems [69], with small test systems already in orbit (e.g. NASA’s Starling satellite swarm [22]). But achieving such rapid analysis is challenging, due to 1) ever-increasing dataset sizes of large-area, high-resolution

^{*}Corresponding author: claganier@turing.ac.uk



This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License.

SEC '25, Arlington, VA, USA

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2238-7/25/12

<https://doi.org/10.1145/3769102.3772713>

¹Code: <https://github.com/alan-turing-institute/sar-vessel-detection-fpga>

imagery (several gigabytes for individual images, missions often producing TB/day [57]), 2) bottlenecks in both transfer of data to ground stations and then on to cloud-based compute (typically hours for each [68]), and 3) the computational complexity of the analysis tasks themselves, which often require accurate extraction, detection or classification of small-magnitude signals in large, high-dimensional, diverse and noisy datasets (e.g. [50]). On-satellite Machine learning (ML) has major potential to reduce the time from data acquisition to analysis, avoiding the latency associated with downlinking full satellite images (e.g. [22]). This latency includes delays of up to several hours for a satellite to pass over a given ground station [70], which cannot be reduced by ground station-based compute, and is challenging to reduce beyond tens of minutes even with deployment of additional multi-million USD ground stations [70] or geostationary data relay satellites (residual latencies of up to ~45 minutes [18] due to visibility between the observing and geostationary satellites). However, on-satellite ML requires careful design and optimization of ultra-efficient processing architectures that can achieve required performance and speed of analysis, whilst also operating within severely restricted power budgets (typically ranging from 2 to 20 watts in CubeSats [31]), with limited computational resources, and on a constrained selection of edge processing hardware for which space-hardening is possible (e.g. FPGAs, ASICs). Therefore, demonstration of ML architectures suitable for on-satellite deployment has to-date only been achieved for a handful of remote sensing applications (e.g. [24, 51, 61, 64]). In this study we tackle this problem for vessel detection with Synthetic Aperture Radar (SAR) satellite data, a critical maritime security application for which on-satellite ML analysis is not yet possible but which exemplifies the deployment requirements and challenges. In particular, the ship detection task is computationally challenging, maritime intelligence is typically required within 30 minutes of collection to be considered actionable [2], and tip-and-cue or other forms of reactive rapid tasking are highly relevant for detection and tracking of moving vessels. SAR vessel detection has previously been demonstrated only by ML models that either are too large for satellite deployment [12, 52, 80, 85, 87], have not been developed for sufficiently low-power hardware [21], or have only been developed and tested on small SAR datasets that do not sufficiently represent the complexity of the real-world task [19, 34, 79]. In this study, we make a major step towards real-world on-satellite deployment of ML models for SAR vessel detection. Our main contributions are:

- We systematically explore a suite of previous architectural adaptations to develop a novel YOLOv8 architecture specifically optimized for SAR vessel detection and FPGA-based processing.

- We deploy our model on a AMD/Xilinx Kria KV260 FPGA, and show it can analyze a 40,000 km² SAR scene (~700 megapixels) in less than a minute, within common satellite power constraints (<10W).
- Our model is between ~50 and ~2500 times more computationally efficient than state-of-the-art GPU-based models, whilst achieving detection and classification performance only ~2% and 3% lower than values from these models on the largest and most diverse open SAR vessel dataset, xView3-SAR [59].

2 Background and Related Work

2.1 Vessel Detection with Synthetic Aperture Radar and Machine Learning

The use of Synthetic Aperture Radar (SAR) imagery to detect “dark” ships (those that disable AIS; Automatic Identification Systems) is critical for maritime surveillance; including for monitoring and combating illegal fishing [41], piracy [63], and trafficking [67]. These applications typically require detection and classification of ships in quasi real-time for rapid response, ideally within minutes of data acquisition [43].

SAR satellites deployed in near-polar low earth orbit are highly suited for this task due to their all-weather, 24-hour, active imaging capabilities, and their combination of large imaging footprint (single image up to several hundred kms wide, e.g. see Fig. 1a-b) with moderate-high resolution (pixel size of meters to 10s of meters) [54]. These capabilities directly relate to how SAR satellites collect data; actively illuminating a large ground area with microwave pulses and recording the complex radar echoes (termed Level 0 or raw data). An initial processing step then leverages frequency modulation of the radar pulse and successive overlapping radar echoes to “focus” the image and increase the ground resolution. The resultant high-resolution images (Level 1 data) contain amplitude and phase data for one or multiple polarizations. Modern SAR satellites such as Sentinel-1, ALOS-4 and NISAR [17, 36, 55] each generate multiple terabytes of imagery every day.

Vessel detection in SAR amplitude images is superficially similar to small object detection in RGB imagery, but aspects of the task and unique data modality make it significantly more challenging. In particular, vessels are much smaller than typical “small objects”; average vessels in our dataset (Fig. 1) are 9 pixels long and have area just 0.002% relative to the size of an 800×800 pixel chip, whilst the smallest vessels are 2 pixels long and have relative area 0.0002%. These relative sizes are ~40–400 times smaller than the lower bound of a commonly-used size definition for small objects in computer vision; 0.08–0.58% of image area [13]. Moreover, the vessels are similar in spatial extent and amplitude to background clutter (e.g. see Fig. 1c-e), the inherent speckle noise

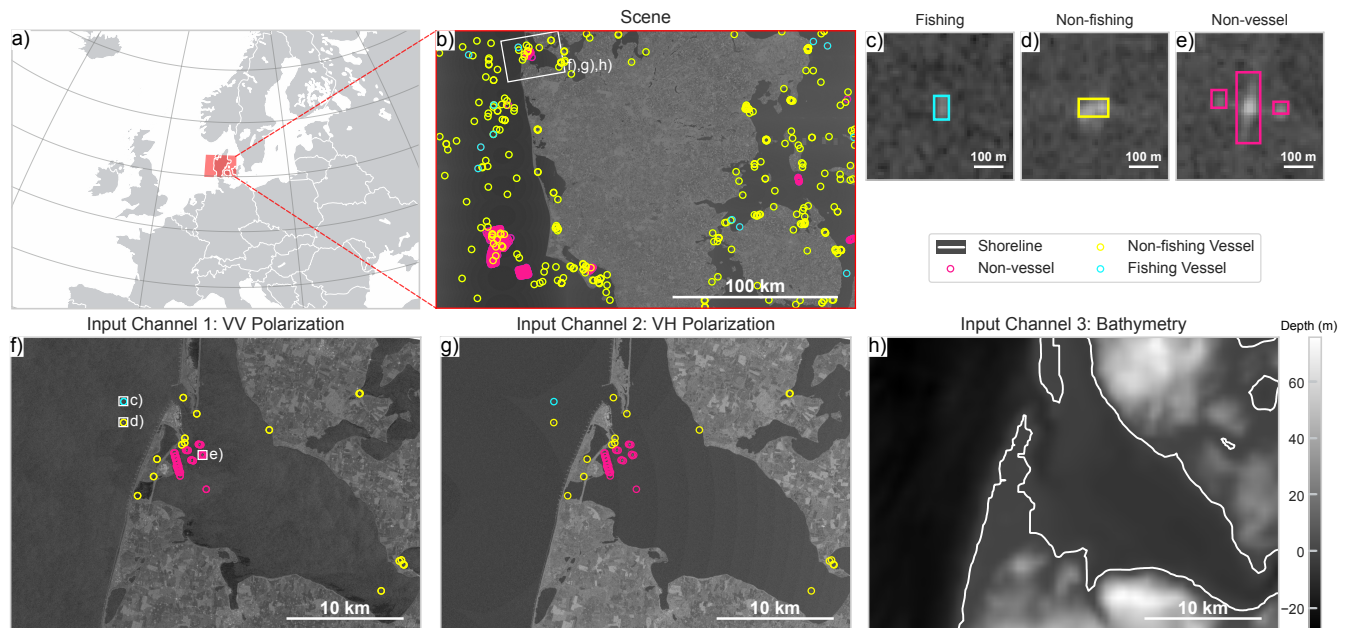


Figure 1: Example imagery from our dataset to illustrate the SAR Vessel Detection Task. (a,b) Single Sentinel-1 SAR scene over Denmark from the xView3-SAR dataset [59]. (b) shows amplitude of the VH band for the full scene with colored circles denoting ground truth fishing vessel (cyan), non-fishing vessel (yellow) and non-vessel (magenta) objects. (c-e) show sample close-up images of fishing vessel (c), non-fishing vessel (d) and non-vessel objects (e). (f-h) show the three channels used as model input in this study, for the highlighted region in (b); VV band SAR amplitude (f), VH band SAR amplitude (g) and bathymetry (h).

in SAR images and high variability in vessel signatures across different sea states further complicates separation of vessels and background [5], and these challenges are especially pronounced in near-shore regions, making the task even harder in these areas [44]. Finally, the input has only two bands (dual-polarization) rather than three (RGB).

Computer vision techniques for SAR vessel detection [88] have addressed the challenges associated with task difficulty, enabling automated, adaptable and robust analysis and improving on accuracy and adaptability over traditional rule-based methods [16, 81]. But although state-of-the-art models achieve high accuracy across diverse maritime conditions (e.g. as represented by the largest and most diverse open SAR vessel dataset; xView3-SAR [59]), this has relied on large-scale architectures and ensemble methods. Such models are therefore too large and power-hungry for satellite deployment (the top 5 performing models from the competition associated with the xView3-SAR dataset have 89M to 754M parameters; 249 MB to 3.0 GB [59]; see blue symbols in Table 2 and Fig. 3). Various lightweight models for SAR vessel detection have also been proposed, using a range of architectures such as LSR-Det [52], SAR-Net [12], and YOLO [80, 85, 87],

but the majority of these models have either not been deployed or evaluated on edge hardware [12, 52, 80, 85, 87], or else have been developed for and tested on platforms with power requirements exceeding satellite constraints [21]. Finally, the only models that have been deployed on hardware with low enough power to meet these constraints (Field-Programmable Gate Arrays; FPGAs [19, 34, 79]) have only been trained and evaluated on relatively small and less representative SAR vessel datasets [71, 72, 83] that do not reflect the real-world scale, diversity, and complexity of operational maritime scenarios captured in large-scale benchmarks such as xView3-SAR (1-2 orders of magnitude more ship instances, 2-3 orders of magnitude more image pixels).

2.2 YOLOv8 for SAR & Small Object Detection

In this study we use the lightweight YOLOv8 architecture [62] (Fig. 2) as our base model, due to its previous success for both SAR vessel detection [29, 48] and small object detection [40] tasks. Here we briefly describe the base architecture and review existing literature on using YOLOv8 for SAR vessel and small object detection. YOLOv8 adds anchor-free detection and has improved accuracy and processing speed

relative to previous versions. It uses a modified CSPDarkNet53 architecture [9] as a feature extractor (Fig. 2a), which incorporates five downsampling operations from $2\times$ to $32\times$ and produces the scaled output feature maps P1-P5. This backbone uses a “C2f” block; an optimized Cross Stage Partial (CSP) block with two Conv modules (*Convolution-BN-SiLU*) and n bottlenecks. The SPPF (Spatial Pyramid Pooling - Fast) module then aggregates multi-scale spatial information in the final stage of the backbone, and the P3, P4, and P5 feature maps (of dimensions 100×100 , 50×50 , and 25×25 , respectively, for a 800×800 pixel input) are passed to the model’s neck. This neck (Fig. 2b) combines these multi-scale feature maps in both a top-down and bottom-up network structure, using a Path Aggregation Network Feature Pyramid Network (FPN) [45, 47]. Finally, the YOLOv8 heads (Fig. 2c) separate object classification (using binary cross-entropy loss) from bounding box regression (using a combination of Distribution Focal Loss (DFL) and Complete IoU (CIoU)).

2.2.1 Modifications for SAR vessels and small objects: Prior work has introduced four major groups of architectural and functional modifications to improve small target detection. First, modified convolutional blocks have been employed to enhance detection performance, to reduce computational complexity, or both. [48] implemented both GhostConv and RepGhost bottlenecks to reduce model size without compromising representational capacity, whereas [11, 75] replaced C2f blocks with C3 blocks, incorporating GhostConv bottlenecks to enhance the efficiency of small object detection. [29] utilized Partial Convolution (PConv) as an alternative approach for improving feature learning efficiency on SAR scenes. Second, FPN modifications can also improve SAR detection. To preserve fine-grained spatial details essential for small target identification, [25, 38] incorporated the P2 feature map, which captures higher-resolution features lost in deeper layers. [25] further added an Asymptotic Feature Pyramid Network (AFPNet) to improve multi-scale feature fusion, while [40] implemented a Generalized Feature Pyramid Network (GFPN) for small-object detection in non-SAR contexts. Third, attention mechanisms have been adopted to improve feature discrimination in cluttered SAR imagery. [48] implemented ShuffleAttention to model channel-wise dependencies, and both [25, 29] employed Efficient Multi-scale Attention (EMA [58]) modules to enhance cross-scale feature representation. [20] demonstrated improved localization of small objects in non-SAR images by applying Coordinate Attention (CA [32]) to embed precise positional information within channel attention. Fourth, alternative loss functions have also been used to enhance localization accuracy for vessel detection, including Wise IoU (WIoU) loss to dynamically adjust focus based on anchor quality through a non-monotonic penalty mechanism [48], and MP-DIoU loss

to improve overlap estimation by minimizing distances between the corners of predicted and ground truth boxes [29]. The Powerful IoU (PIoU) loss has also been used to reduce localization error for non-SAR small-objects [40], minimizing the Euclidean distance between bounding box corners.

3 Methodology

In Section 3.1 we describe our proposed model architecture, which employs a tailored set of modifications to adapt the YOLOv8 model to our specific task. We then detail our FPGA deployment in Section 3.2, followed by an overview of the xView3-SAR dataset in Section 3.3. Finally, we describe our evaluation metrics in Section 3.4 and the training setup for our models in Section 3.5.

3.1 Proposed Architecture

Our proposed architecture represents a novel combination of three of the modifications outlined in the previous section, to enhance SAR vessel detection performance and computational efficiency. In our experiments we systematically tested a broader set of the modifications discussed in the previous section; for brevity we present illustrative results showing why they were not incorporated in our final architecture in Table 2 and briefly discuss these in Section 4.1.

First, to better leverage multi-scale feature representations, we extend the FPN by incorporating the P2 feature map and an additional detection head (Fig. 2). Higher-resolution feature maps (P1, P2) better capture fine-grained details useful for detecting small targets, whilst deeper features (P4, P5) are more effective at detecting larger structures.

Second, we introduce two architectural improvements to enhance performance while reducing computational overhead. We replace the C2f blocks with CSP blocks featuring three Conv modules (referred to as C3), which incorporate an additional Conv module and enhance feature representation with minimal computational overhead. We also substitute the up- and down-sampling Conv blocks in the backbone and neck, as well as those in the bottleneck modules, with GhostConv [28]. This lightweight convolution variant reduces both computational and parameter overhead by generating feature maps through a two-stage process; a primary convolution produces a low-dimensional representation ($\lfloor c_{out}/2 \rfloor$ channels), which is then concatenated with the output of a secondary depthwise convolution of matching dimensions.

Third, to address the inherent challenges of small object detection, we replace the standard IoU loss with PIoU2 [46]. This modified loss function features an adaptive penalty factor based on target size (calculated from the average absolute distance between corresponding edges of prediction and ground truth boxes) and a non-monotonic focusing mechanism that concentrates learning on bounding boxes with

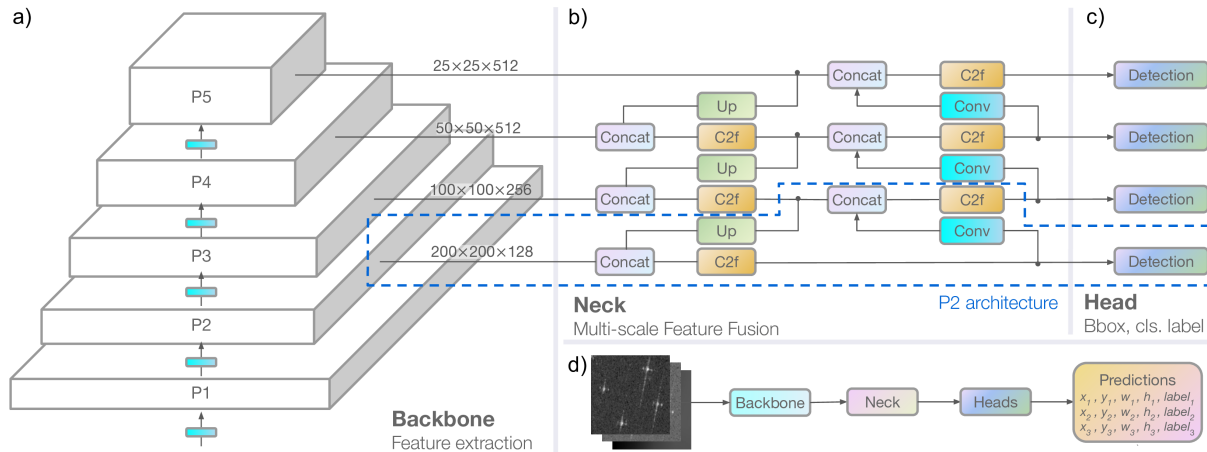


Figure 2: Architecture diagram of the YOLOv8-based object detection model. (a) feature extraction backbone, (b) multi-scale feature fusion neck, (c) detection head, (d) simplified end-to-end detection pipeline from SAR and bathymetry input to predictions. P2 architecture branch is shown in dashed blue box. Diagram modified from [60].

moderate prediction quality. This approach has not previously been used for SAR vessel detection.

3.2 FPGA Deployment

In this study we specifically investigate FPGA deployment, due to their power efficiency and existing widespread use on-board satellites. We focus on a single FPGA device as the one selected fits within the strict power constraints of a representative small satellite platform. We evaluate our proposed solution on the AMD/Xilinx Kria KV260 board, which integrates programmable logic (PL; Table 1) with a processing system (PS) based on a Quad-core Arm Cortex-A53 SoC. For model deployment, we utilized the Vitis AI 3.5 framework and created a custom hardware design targeting the PL of the Kria KV260 and incorporating the Deep Learning Processing Unit (DPU) 4.1 IP using Vivado 2023.2. The DPU is a programmable engine optimized for deep neural networks, which is leveraged by the Vitis AI instruction set to run models in the PL. In our design, the DPU is configured with one B4096 core and a clock frequency of 300 MHz. We quantized the model to INT8 using the Vitis AI Quantizer library, using symmetric quantization with a power-of-two scaling scheme (required configuration for the DPU), allowing for both Post-Training Quantization (PTQ) and Quantization-Aware Training (QAT) using a straight-through estimator [35]. The DPU of the Kria KV260 platform (DPUCZDX8G) does not support the SiLU activation function used in YOLOv8, and so we modified the model to use Hard Swish activations [33] instead. Finally, we deployed the compiled model on the FPGA using the Vitis AI Runtime, and evaluated model performance in terms of speed, power consumption, and accuracy.

Table 1: FPGA resource utilization summary.

LUT	FF	DSP	BRAM	URAM
<i>Kria KV260</i>				
117,129	234,240	1,248	144	64
<i>Vitis AI B4096 DPU</i>				
62,104	123,486	710	109	40
(53.0%)	(53.2%)	(56.9%)	(75.7%)	(62.5%)
<i>Post-Processing Decoding Kernel</i>				
45,671	32,011	188	33	0
(39.0%)	(13.7%)	(15.1%)	(22.9%)	(0.0%)

3.3 Dataset

The xView3-SAR [59] dataset is a large and diverse collection of high-resolution SAR maritime images, including 754 publicly available images with an average size of 29,400 by 24,400 pixels. The scenes are C-band SAR amplitude images acquired by the European Space Agency's (ESA's) Sentinel-1 constellation; specifically the Interferometric Wide Swath mode Level-1 Ground Range Detected product, which includes both vertical-horizontal (VH) and vertical-vertical (VV) polarization bands for each image at a 10-meter pixel spacing. The scenes are supplied having undergone pre-processing, including orbit correction, removal of noise artifacts, radiometric calibration, terrain correction, and geocoding. In addition to the VH and VV SAR images, ancillary data are provided including wind speed, direction, quality, and land/ice mask information from Sentinel-1 Level-2 Ocean product, and bathymetry information from the General Bathymetric Chart of the Oceans (GEBCO, [1]). These

layers are provided in the xView3-SAR dataset co-registered with the respective SAR scenes and at a lower (500-meter) resolution than the SAR images. The scenes are distributed over four distinct geographical regions in European waters (Adriatic, Bay of Biscay, Iceland, and the North Sea) and one in West African waters (Gulf of Guinea).

The xView3-SAR competition dataset provides labels for maritime object detections derived from two sources: the AIS database and manual expert reviewers. AIS is a radio transponder system that broadcasts ships' GNSS coordinates. The labels include geographic/pixel coordinates, class, bounding box coordinates and dimensions, length, distance from shore and confidence score. Class labels include non-vessels (fixed infrastructure such as wind farms, fish cages, platforms, and port towers), fishing vessels and non-fishing vessels. An example scene is shown in Fig. 1a,b.

The dataset is divided into training (554 scenes), validation (50), and test sets (150). The training set contains automatically labeled (i.e. AIS) detections only, while the others contain both expert-reviewed AIS detection and manually-labeled detections. Consequently, the training dataset lacks manually annotated bounding boxes and is generally of lower quality, with occasional mismatches between bounding boxes and detected objects caused by the automated matching process. Additionally, some fixed infrastructure is missing ground-truth labels, and because many vessels do not broadcast AIS signals in near-shore areas, the training set contains approximately 100× fewer near-shore detections as a proportion of total detections than the validation set.

3.4 Evaluation Metrics

To evaluate our models we use the test dataset, which contains 60,010 detections, and four detection and classification metrics from the xView3-SAR competition:

- (1) Detection F1 score, evaluating identification of maritime objects across different sea states and geographical areas.
- (2) Near-shore F1 score, evaluating detection of maritime objects that are within two kilometers from shore (as defined by the zero-point in co-registered bathymetry data), in typically higher vessel density areas.
- (3) Vessel Classification F1 score, evaluating performance at identifying whether a maritime object is a “vessel” or “fixed infrastructure” (non-vessel). This classification is only performed on manual labels with a high or medium confidence score, or that are identified via correlation with AIS.
- (4) Fishing Classification F1 score, evaluating the model's performance in identifying fishing and non-fishing vessels. This classification is evaluated with AIS labels.

This evaluation allows for direct comparison and benchmarking of our model to the models from the top-5 winning entries to the xView3-SAR competition [59]. The evaluation metrics for these models are reported in the top section of Table 2 (blue symbols). The trained model weights and code for these models were also made available by competitors, as well as for the competition reference model (Faster R-CNN ResNet50). The competition also provides a vessel length estimation metric which is not used in this work, as our focus is on detection and classification performance.

3.5 Training Setup

Our training methodology employs two different datasets and two distinct training strategies to generate our final model. We utilize both the xView3-SAR training *and* validation sets for training; the latter offers higher-quality annotations (see section C above), and has demonstrated superior detection performance when used as a training set in prior competition-winning solutions and in our own experiments. We therefore partitioned the xView3-SAR validation set into 40 scenes for training and 10 scenes for validation. These subsets are referred to as the *training* and *validation* sets throughout this work; the former is used for our initial exploratory analysis and for fine-tuning our final models. Additionally, we addressed the limitations of the automatically-generated labels in the original xView3-SAR training set by generating pseudo-labels and bounding boxes using our best-performing exploratory model (YOLOv8n-Ghost-P2-PIoU2 trained for 100 epochs). This enhanced dataset is referred to hereafter as the *pre-training* set.

For initial model exploration (Table 2.A-D), we trained models on the *training* set for 100 epochs using AdamW (batch size: 16, learning rate: 1.429×10^{-3} , momentum: 0.9, linear warm-up: 3 epochs). For the final model development (Table 2.E), we adopted a two-stage training procedure inspired by the transfer learning literature [76, 86] and successful competition entries. This process involved first learning broadly transferable representations from a large, diverse, and potentially noisy dataset, followed by fine-tuning the model on a smaller dataset with clean labels for specialization. In the pre-training stage (**PT**), the model was trained for 100 epochs on the pseudo-labeled *pre-training* dataset using AdamW (batch size: 32, learning rate: 7.14×10^{-4} , momentum: 0.9, linear warm-up: 3 epochs). This was followed by a fine-tuning stage (**FT**), in which the model was trained for another 100 epochs on the *training* set using stochastic gradient descent with momentum (batch size: 16, learning rate: 5×10^{-3} , momentum: 0.9, no warm-up). Finally, we applied a threshold adjustment strategy (**TH**), in which the prediction threshold varies by predicted class label (non-vessel, non-fishing vessel, fishing vessel) and proximity to shore (within

or outside 2 km). We optimized thresholds on the *validation* set via grid search, selecting the configuration with the highest detection F1 score. We then fixed these thresholds and applied them unchanged to the held-out test set.

We trained all models using the Ultralytics codebase with PyTorch 2.4.1 and CUDA 12.1 on an NVIDIA A100 GPU (40 GB). The model input consisted of three channels; VV and VH SAR bands along with bathymetry data (e.g. see Fig. 1f-h and Fig. 2d), with scenes partitioned into 800 by 800 pixel chips using a tiling approach. SAR channel amplitudes were clipped to the range [-50, 20] dB, while bathymetry values were clipped to [-6000, 2000] meters, with each channel subsequently linearly normalized to the range [0, 255] for model input. We incorporated negative samples (background chips containing no detections) into the training data after identifying that this approach improved overall model detection performance by up to 2.9% (see study in Table 3.A). Consequently, we implemented a 20% background ratio in the model training data, with background chips randomly sampled from the processed dataset. To enhance model generalization, we implemented standard image augmentations, including mosaic, random translations, scaling, left-right flips, image blurring, and hue-saturation-value adjustments.

4 Results

We present the results of training the different YOLOv8 architectures in Section 4.1, the evaluation of the quantized model in Section 4.2, and the performance of the deployed model running on FPGA hardware in Section 4.3.

4.1 Model Modification Results

The model size, computational complexity and performance on the test dataset for a representative range of the various model architectures is presented in Table 2. Our analysis of YOLOv8 model sizes (n , s , m and l) demonstrates that smaller architectures consistently outperform larger ones across most metrics (Table 2.A). The smallest model (*nano*) shows improvements of 2.4% in overall detection and 7.4% in near-shore detection over the largest variant, has vessel classification accuracy within 0.3% of larger models and outperforms larger models by up to 1.4% in fishing classification. This inverse relationship between model complexity and performance (also seen by [42]) may suggest that the maritime detection task benefits from simpler, more generalizable feature representations that avoid overfitting to training data complexities and AIS location errors.

Building upon the *nano* baseline architecture, we explored efficiency-enhancing modifications that could maintain performance while reducing computational requirements. The integration of Ghost convolutions into the bottleneck layers (Table 2.B), delivers a 45% reduction in model size and

FLOPs while improving fishing classification performance by 1.5% at the cost of a 0.3% drop in overall detection. Alternative efficient convolutional blocks such as EfficientNet, ShuffleNet, and FasterNet [14, 49, 66] achieved less substantial reductions in model size (ranging from 1.8-2.6M parameters compared to Ghost's 1.7M) while delivering comparable or slightly inferior performance, indicating that Ghost convolutions provide the most effective balance of efficiency and detection performance for this task.

The performance gains from architectural efficiency are further enhanced through multi-scale feature representation. Incorporating higher-resolution P2 feature maps into the *nano* Ghost architecture improves near-shore detection performance by 7.6% compared to the baseline YOLOv8n-Ghost model (Table 2.C). While adding the P2 features increases computational complexity by 72.5%, the Ghost convolution's gains in efficiency offset this increase, keeping the final FLOPs equivalent to the baseline YOLOv8n model. Ablation studies confirm the importance of multi-scale features, as removing the P5 feature map (YOLOv8n-Ghost-P2-3-4) results in performance degradation of 0.7% and 0.4% in overall detection and fishing classification respectively compared to the P2 architecture.

Building on these architectural improvements, we examined loss function variants specifically designed for small object detection. The PIoU2 provides consistent improvements of 1.4% in overall detection and 0.7% in fishing classification relative to the baseline CIoU (Table 2.D). In contrast, using MPDIoU loss results in improvements of 0.3% and 1.3% in classification but a decline in overall detection performance, while WIoU shows performance drops across all metrics compared to CIoU.

From the results of these architecture evaluation experiments, the YOLOv8n-Ghost-P2-PIoU2 model is selected as the best performing architecture; it combines the smallest base model size, the efficiency improvements of the Ghost blocks, the improved detection performance with the addition of the P2 feature map and the improved accuracy of the PIoU2 loss function. We further enhanced the model through the two-stage training approach as discussed in Section 3.5 (Table 2.E). Pre-training on pseudo-labeled data (**PT**) consistently improves fishing classification performance due to increased sample diversity. This comes at the cost of reduced near-shore detection performance, likely caused by fewer AIS-broadcasting vessels in coastal areas, and therefore fewer ground-truth labels, but the subsequent fine-tuning stage on the *training* set (**FT**) recovers these losses while maintaining classification gains. Overall this results in average improvements of 1.0% in detection and 3.4% in classification compared to single-stage training. Notably, following the same training procedure, the baseline unmodified YOLOv8n model trails our final model by 6.5% and 1.3% in detection

Table 2: Performance of YOLOv8 detection models on the xView3-SAR Dataset. Subheadings indicate the specific model architecture and training variations explored. Architectures highlighted in bold represent the selected configurations carried forward to the final model (★). xView3-SAR competition models are listed at the top, with the number of models in ensembles in parentheses. Colored symbols correspond to models illustrated in Fig. 3. F1 scores for subsections A-E are provided as means and standard deviations over three random seeds.

Model	Param (M)	Size (MB)	GFLOPs	F1 Score			
				Detection	Near-Shore	Vessel	Fishing
xView3-SAR Competition [59]							
● CircleNet (12)	322.5	1294.0	8766.9	0.755	0.525	0.942	0.834
■ UNet (6)	432.5	1732.0	21079.1	0.752	0.473	0.938	0.803
◆ HRNet (15)	754.3	3029.0	11802.8	0.730	0.467	0.932	0.828
▼ Faster R-CNN	89.1	249.0	5889.6	0.726	0.428	0.954	0.822
▲ HRNet (12)	328.1	1314.0	426.5	0.725	0.461	0.940	0.797
A) Model Size							
YOLOv8l	43.7	175.4	165.2	0.607±0.006	0.293±0.020	0.895±0.014	0.684±0.021
YOLOv8m	25.9	104.1	78.9	0.627±0.014	0.364±0.009	0.913±0.004	0.679±0.008
YOLOv8s	11.2	45.1	28.6	0.622±0.007	0.362±0.006	0.915±0.001	0.684±0.010
YOLOv8n	3.2	12.6	8.7	0.631±0.008	0.367±0.017	0.912±0.006	0.693±0.009
B) Convolutional Block							
YOLOv8n-EfficientNet	2.6	11.4	7.3	0.630±0.003	0.367±0.013	0.911±0.003	0.708±0.006
YOLOv8n-ShuffleNet	1.8	7.6	5.3	0.626±0.002	0.352±0.002	0.904±0.004	0.685±0.002
YOLOv8n-FasterNet	1.9	8.4	5.8	0.628±0.005	0.359±0.003	0.912±0.003	0.704±0.007
YOLOv8n-Ghost	1.7	7.6	5.1	0.628±0.002	0.361±0.007	0.912±0.001	0.708±0.011
C) P2 Feature Map							
YOLOv8n-Ghost-P2-3-4	1.1	4.6	8.3	0.664±0.003	0.435±0.005	0.919±0.003	0.701±0.003
YOLOv8n-Ghost-P2	1.6	7.8	8.8	0.671±0.002	0.437±0.004	0.918±0.003	0.705±0.006
D) Bounding Box Loss							
YOLOv8n-Ghost-P2-MPDIoU	1.6	7.8	8.8	0.664±0.002	0.437±0.002	0.921±0.006	0.718±0.010
YOLOv8n-Ghost-P2-WIoU	1.6	7.8	8.8	0.670±0.004	0.420±0.025	0.908±0.014	0.697±0.022
YOLOv8n-Ghost-P2-PIoU2	1.6	7.8	8.8	0.685±0.001	0.436±0.004	0.917±0.004	0.712±0.008
E) Two-Stage Training							
YOLOv8n (PT)	3.2	12.6	8.7	0.621±0.001	0.289±0.004	0.873±0.003	0.693±0.008
YOLOv8n (FT)	3.2	12.6	8.7	0.632±0.007	0.379±0.011	0.928±0.002	0.741±0.013
YOLOv8n (FT+TH)	3.2	12.6	8.7	0.650±0.002	0.385±0.008	0.918±0.003	0.737±0.016
YOLOv8s-Ghost-P2-PIoU (PT)	5.3	22.6	22.3	0.692±0.002	0.328±0.007	0.869±0.001	0.756±0.002
YOLOv8s-Ghost-P2-PIoU (FT)	5.3	22.6	22.3	0.694±0.003	0.448±0.014	0.932±0.002	0.769±0.011
● YOLOv8s-Ghost-P2-PIoU (FT+TH)	5.3	22.6	22.3	0.703±0.004	0.444±0.010	0.928±0.003	0.770±0.013
YOLOv8n-Ghost-P2-PIoU (PT)	1.6	7.8	8.8	0.686±0.001	0.324±0.002	0.865±0.003	0.743±0.003
YOLOv8n-Ghost-P2-PIoU (FT)	1.6	7.8	8.8	0.691±0.003	0.450±0.003	0.929±0.003	0.767±0.006
● YOLOv8n-Ghost-P2-PIoU (FT+TH)	1.6	7.8	8.8	0.704±0.001	0.447±0.007	0.925±0.003	0.769±0.008
F) FPGA							
★ YOLOv8s-Ghost-P2-PIoU2	5.3	5.1	37.2 [*]	0.697	0.438	0.931	0.785
★ YOLOv8n-Ghost-P2-PIoU2	1.6	1.5	14.6 [*]	0.704	0.445	0.921	0.765

*INT8 operations (OPs) on FPGA DPU are reported instead of FLOPs due to model quantization.

and classification respectively. Finally, with the threshold adjustment strategy (TH) we are able to increase overall detection performance by 1.3% on our YOLOv8n-Ghost-P2-PIoU2 model, with a 0.4% average drop in the other metrics.

Comparing the performance of our best models to the top-5 winning entries of the xView3-SAR competition (see Fig. 3), our model nears the competition models with between 98.2% and 99.8% fewer parameters. Our detection performance is

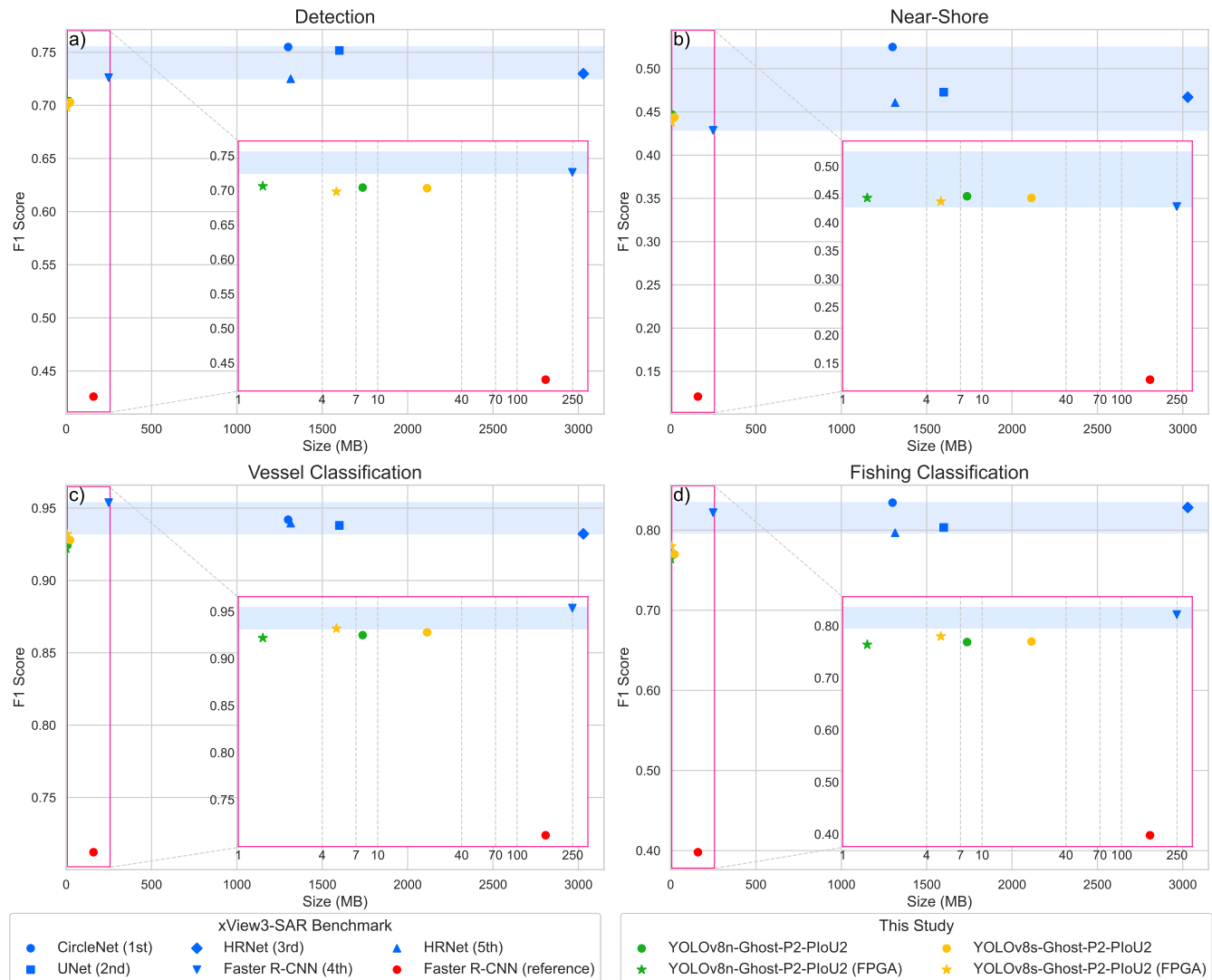


Figure 3: F1 Score vs. Model Size (MB) for Detection (a), Near-Shore Detection (b), Vessel Classification (c), and Fishing Vessel Classification (d) tasks. In each plot, xView3-SAR competition models are shown in blue, with the range of their F1 scores shown by the pale blue band. The competition reference model is shown by the red circle. Models from this study are shown in yellow (YOLOv8 *small*) and green (YOLOv8 *nano*), where floating point and quantized, FPGA-deployed models are denoted by circles and stars respectively. Inset plots in pink show a zoomed-in view of each corresponding plot between 0 and 250 MB on the x-axis (logarithmic scale).

within 2.1% of the top-5 winning entries and outperforms one model in near-shore detection by 1.9%. Our model is only 0.7% and 2.8% behind the top-5 in vessel and fishing classification respectively. We also investigated several additional modifications, following methods from the literature, which did not show improvements for our implementation. Modifying the model's FPN to AFPN or GFPN showed a performance drop across the different metrics, with slight

model size increases. Similarly, incorporating attention modules (Convolutional Block Attention Module [74], CA, EMA) at the point where feature maps are fed into the FPN improved fishing classification performance by an average of 2.8%, though this came at the cost of reduced overall detection performance. Our proposed model is substantially more computationally efficient than existing methods (8.8 GFLOPs rather than 426.5–21079.1). This corresponds to a 98.0% to

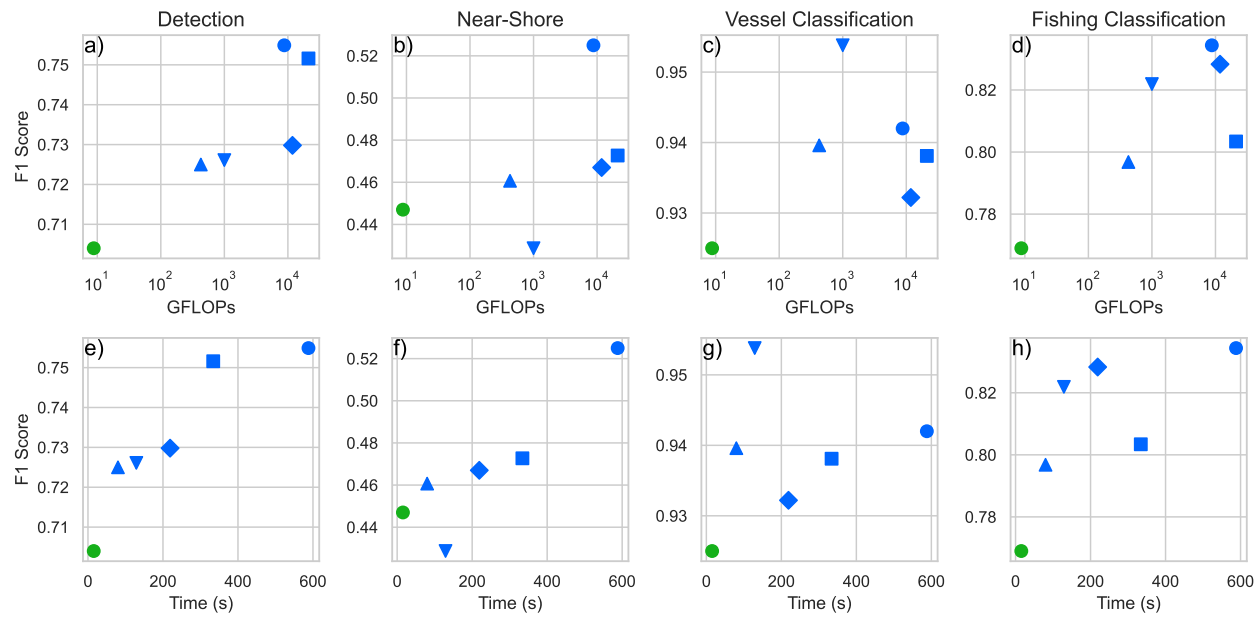


Figure 4: F1 Score vs. Model complexity (a-d; GFLOPs, logarithmic scale) and vs. Average inference time per scene (e-h; s, over three scenes) for Detection, Near-Shore Detection, Vessel and Fishing Vessel Classification. xView3-SAR competition top-5 models shown in blue and our YOLOv8n-Ghost-P2-PIoU2 model in green (see legend in Fig. 3).

99.9% reduction in computational complexity, without performance loss beyond observed efficiency-performance trends (Fig. 4.a-d). This efficiency is reflected by inference speed on an NVIDIA A100 GPU, where our model processes a scene in 15.4 seconds on average, achieving a $5.2\times$ to $38.2\times$ speed-up over alternatives (80 to 588 seconds; Fig. 4.e-h). The magnitude of the speed-up is less pronounced than FLOPs reduction, as the GPU's architecture is highly effective at parallelizing the workloads of the more computationally intensive models, and our model is not optimized for GPU.

Table 3: Impact of the Background Ratio and Auxiliary Channel on YOLOv8n-Ghost-P2-PIoU2 performance.

Metric	F1 Score			
	Detection	Near-Shore	Vessel	Fishing
<i>A) Background Ratio</i>				
40	0.680 ± 0.015	0.433 ± 0.005	0.917 ± 0.004	0.711 ± 0.011
30	0.684 ± 0.001	0.436 ± 0.004	0.917 ± 0.002	0.706 ± 0.005
20	0.685 ± 0.001	0.436 ± 0.004	0.917 ± 0.004	0.712 ± 0.008
10	0.672 ± 0.007	0.417 ± 0.012	0.914 ± 0.004	0.705 ± 0.022
0	0.656 ± 0.016	0.416 ± 0.017	0.915 ± 0.003	0.701 ± 0.018
<i>B) Auxiliary Channel</i>				
Bath	0.685 ± 0.001	0.436 ± 0.004	0.917 ± 0.004	0.712 ± 0.008
Ø	0.690 ± 0.007	0.423 ± 0.017	0.907 ± 0.013	0.694 ± 0.014

4.2 Quantization Results

We employ quantization to reduce model size and computational complexity, as well as to meet the constraints of the target FPGA hardware. As the SiLU activation is not supported by the framework, we replace it with Hard Swish and finetune (FT) the model, with negligible deterioration to the float model (Table 5.A). We evaluate quantized models on the test dataset on GPU, and Table 4 compares performance across our YOLOv8 variants, all trained with two-stage training and modified activation.

4.2.1 Post-Training Quantization (PTQ). We calibrated our models on a 1,000 chip subset of the *validation* set. Smaller (100) and larger (10,000) calibration sets were also tested, but these changes did not result in significant performance differences. With PTQ, models show moderate performance degradation compared to their floating-point counterparts, with a minor overall detection drop of 0.4% and a decrease of an average of 3.4% in near-shore and classification metrics. The baseline YOLOv8n model experiences the largest performance drop, with an average performance drop of 4.1% across all metrics, compared to 1.7% and 2.2% for our modified *small* and *nano* models respectively. These results indicate that quantization exacerbated the limitations of the less performant baseline architecture, while our modified architectures are more robust to quantization. Additionally,

the increased performance drop of the smallest model (*nano*) can be attributed to the reduced amount of redundancy with fewer parameters, which leads to a more pronounced impact.

Table 4: Performance of YOLOv8 model variants with INT8 PTQ and QAT on GPU, and QAT on FPGA.

F1 Score				
Model	Detection	Near-Shore	Vessel	Fishing
A) YOLOv8n (PT+FT+TH)				
Float	0.650±0.003	0.384±0.003	0.914±0.003	0.731±0.007
PTQ	0.644±0.003	0.327±0.012	0.872±0.016	0.671±0.012
QAT	0.646±0.004	0.377±0.011	0.915±0.002	0.725±0.018
FPGA	0.647	0.386	0.914	0.730
B) YOLOv8s-Ghost-P2-PIoU2 (PT+FT+TH)				
● Float	0.701±0.001	0.442±0.003	0.920±0.018	0.774±0.013
PTQ	0.696±0.005	0.420±0.015	0.903±0.010	0.752±0.013
QAT	0.699±0.001	0.444±0.007	0.932±0.001	0.779±0.001
★ FPGA	0.697	0.438	0.931	0.785
C) YOLOv8n-Ghost-P2-PIoU2 (PT+FT+TH)				
● Float	0.699±0.004	0.444±0.004	0.923±0.002	0.769±0.006
PTQ	0.697±0.005	0.419±0.009	0.892±0.004	0.740±0.004
QAT	0.701±0.003	0.445±0.001	0.927±0.005	0.771±0.009
★ FPGA	0.704	0.445	0.921	0.765

4.2.2 Quantization-Aware Training (QAT). To minimize the performance drop observed with PTQ, we modify the second training stage (FT) with QAT. We use the same training parameters as the fine-tuning stage detailed in Section 3.5, with the addition of a quantizer parameter learning rate. We study the impact of the quantizer learning rate on the model performance (Table 5) and signal-to-quantization-noise ratio (SQNR; Fig. 5) of the model subgraphs (partitions of the computational graph). The SQNR provides a measure of the fidelity of the quantized representation, where higher values correspond to reduced distortion due to quantization and is defined as $SQNR = 10 \cdot \log_{10}(\frac{P_{\text{signal}}}{P_{\text{noise}}})$. We find that higher learning rates (5 and 0.5) produce a broader distribution, with higher SQNR modes (23 and 24 dB) with a longer tail towards lower SQNR values. Lower learning rates (0.05 and 0.005) produce a more concentrated distribution, with a lower SQNR mode (20 dB) and fewer low SQNR subgraphs. However, the detection and classification performance difference is minimal across the different values, and we therefore opt for a learning rate of 0.5 in our solution. QAT implementation introduces minimal performance degradation across most metrics (average -0.4% for detection F1), with our modified architecture models showing slight improvement in classification metrics (e.g. +0.9% for YOLOv8s-Ghost-P2-PIoU2 QAT). The baseline architecture shows the highest average

performance drop of 0.7%, with the *nano* modified architectures showing a loss of 0.1% and the *small* a 0.2% improvement overall. Nevertheless, the YOLOv8n-Ghost-P2-PIoU2 model shows the highest detection performance overall, with a 0.701 and 0.445 F1 score for detection and near-shore detection respectively, outperforming the baseline model and the *small* model by 5.5% and 0.2% respectively. These results indicate that with QAT, most of the performance drop observed with PTQ can be recovered, and confirms that our two-stage training approach effectively improves model performance.

Table 5: Impact of Activation and Quantizer Learning Rate on YOLOv8n-Ghost-P2-PIoU2 performance.

F1 Score				
Metric	Detection	Near-Shore	Vessel	Fishing
A) Activation Function (FT)				
SiLU	0.691±0.003	0.450±0.003	0.929±0.003	0.767±0.006
HSwish	0.686±0.005	0.446±0.003	0.928±0.003	0.766±0.006
B) Quantizer Learning Rate (QAT FT)				
5	0.690±0.004	0.443±0.002	0.929±0.002	0.765±0.007
0.5	0.693±0.005	0.443±0.001	0.930±0.001	0.763±0.003
0.05	0.686±0.010	0.439±0.006	0.931±0.003	0.765±0.002
0.005	0.688±0.003	0.444±0.013	0.929±0.008	0.765±0.008

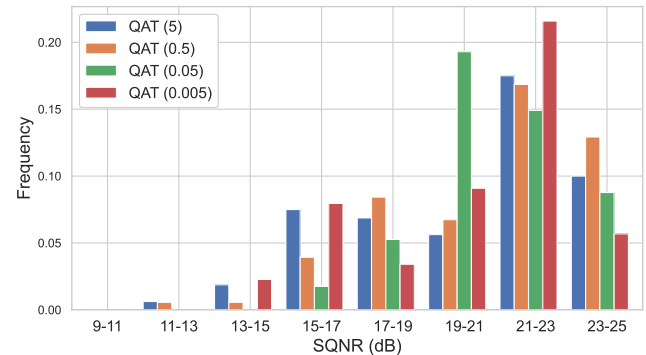


Figure 5: Histogram of per-subgraph SQNR for different QAT learning rates for YOLOv8n-Ghost-P2-PIoU2.

4.3 FPGA Deployment Results

The performance of the deployed compiled QAT models on the Kria KV260 board is shown in Table 4 and matches the expected performance based on the GPU results, with little to no performance degradation observed. For our final model, we also visually illustrate the quality of the results with respect to a state-of-the-art GPU-based model. From

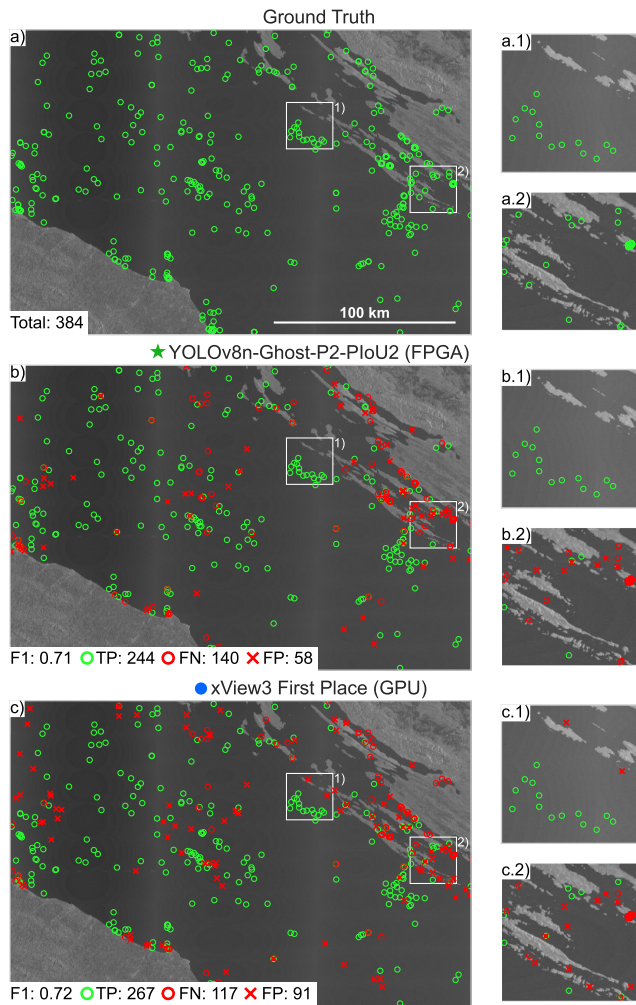


Figure 6: Example scene from xView3-SAR comparing (a) ground truth, (b) predictions from our YOLOv8n-Ghost-P2-PIoU2 model on FPGA, and (c) the first-place competition entry. Insets highlight areas of interest.

simple visual inspection of these qualitative results (Fig. 6), our FPGA-deployed model shows similar levels of performance to the xView3-SAR first-place model. We note that the winning model shows slightly higher performance overall by missing fewer detections (false negatives), at the cost of a larger number of false positives. The winning model also shows slightly higher detection performance in near-shore areas, where our model misses more detections (contrast Fig. 6.b.2 vs c.2). However, it is important to note that our model is only 0.15% of the size of the winning model (Table 2).

The power usage, efficiency, throughput and external memory average bandwidth of the YOLOv8 model variants implemented on the Kria KV260 development board are presented in Table 6, where we provide performance of both

the ‘Full’ pipeline implementations (including pre and post-processing) and of inference alone (‘DPU’). SoC power measurements were collected using `xlntx_platformstats` monitoring software with a 30-second sampling window and samples taken every 0.01 seconds (100 Hz sampling rate). Average power consumption was calculated over the full inference period, while peak power represents the highest reading recorded during operation. When idle, the SoC has a baseline consumption of 4.1 watts.

Table 6: Power, throughput (Tput), full implementation average power efficiency (Full Eff), and average memory bandwidth (BW) of YOLOv8 variants inference on Kria KV260 vs. thread count (1, 2, 4 and 8).

(#)	Power (W)		Tput (FPS)		Full Eff (FPS/W)	BW (MB/s)	
	Avg	Peak	Full	DPU		Full	DPU
A) YOLOv8n							
1	5.4	8.9	6.9	24.7	1.3	4652	4650
2	6.4	9.4	12.8	29.8	2.0	4392	4570
4	7.6	9.8	19.8	29.8	2.6	3664	4571
8	7.8	9.8	21.6	29.8	2.8	3442	4570
B) YOLOv8n-Ghost							
1	5.1	8.6	7.2	27.0	1.4	4375	4374
2	6.0	9.2	13.4	33.6	2.2	4151	4319
4	7.2	9.4	20.9	33.6	2.9	3606	4319
8	7.3	9.7	22.9	33.6	3.1	3372	4318
C) YOLOv8n-Ghost-P2							
1	4.9	9.5	3.6	13.3	0.7	4701	4692
2	5.7	10.0	7.0	20.5	1.2	4564	4599
4	6.9	10.5	11.8	20.5	1.7	4223	4601
8	7.0	10.8	13.4	20.5	1.9	4033	4603
D) YOLOv8s-Ghost-P2							
1	5.3	9.8	3.2	9.3	0.6	5259	5259
2	6.5	10.2	6.2	12.2	1.0	5196	5203
4	8.1	10.4	10.4	12.2	1.3	4710	5202
8	8.1	10.4	10.5	12.2	1.3	4692	5203

Architectural modifications to the YOLOv8n model yield distinct trade-offs on the FPGA platform. The integration of Ghost blocks provides a 10.5% increase in DPU-only throughput, while reducing the end-to-end power consumption and average DPU memory-bandwidth by 5.9%, confirming the efficiency improvement of our architecture on hardware. In a single-threaded environment, the end-to-end throughput across all models is on average 3.5 times lower than the DPU-only inference capabilities, highlighting a severe CPU bottleneck. Increasing the CPU thread count mitigates this limitation, increasing throughput on average by a factor of 3.2 at the cost of increased average and peak power consumption, but performance gains plateau as the system becomes DPU-bound. For YOLOv8n-Ghost, throughput scales

to 22.9 FPS with eight threads, approaching its DPU ceiling of 33.6 FPS. The P2 architecture models are more computationally expensive, with the 14.6 GOPs workload (Table 2) of the YOLOv8n-Ghost-P2 model nearly double the YOLOv8n-Ghost's workload (7.9 GOPs). Whilst the YOLOv8n-Ghost-P2 model has a similar workload to that of the baseline YOLOv8n model (12.7 GOPs), its DPU-only throughput is 46% and 31% lower with one and two threads respectively. This discrepancy highlights a DPU-memory bottleneck, which is caused by its 52% higher external memory load size from additional high-resolution feature maps. However, the P2 models still shows the largest end-to-end throughput improvements from one to eight threads, with the *nano* and *small* models improving by a factor of 3.7 and 3.3 respectively. Analysis of memory bandwidth also shows that as thread count increases, the full-pipeline average bandwidth decreases (e.g. from 4375 to 3372 MB/s for YOLOv8n-Ghost), while the DPU-only bandwidth remains constant. This indicates significant memory bus contention between the parallel CPU threads and the DPU, which throttles the DPU's data access. Similarly, throughput of DPU-only implementations does not increase past two threads, suggesting that threadcount could be used to balance performance against deployment power constraints.

A breakdown of the inference time for different YOLOv8 model architectures is shown in Fig. 7. Image loading and pre-processing requires on average 71 ms, while DPU inference times vary by architecture. The baseline *nano* YOLOv8 model has an average inference time of 33 ms. The Ghost variant achieves a 9% reduction in latency, with an average of 30 ms, attributed to a 41% decrease in computational complexity. However, the speed-up is partially offset by increased memory access overhead due to higher read/write operations. The P2 variant is 60% slower (48 ms) due to its additional layers and larger feature maps. Finally, the larger *small* Ghost P2 model is the slowest at 84 ms. The post-processing step takes on average 41 ms for the baseline architectures and 159 ms for the P2 variants. The increased computation time results from the P2 feature map generating 40,000 additional predictions compared to the baseline configuration. Post-processing consists of two stages: a computationally intensive decoding step that transforms raw outputs into bounding box coordinates, class labels and confidence scores; and Non-Maximum Suppression (NMS) filtering (only takes 91 μ s on average).

5 Discussion

In this study, we have made a key contribution towards on-satellite deployment of ML models for SAR vessel detection. We now discuss further steps towards real-world usage of our model, in terms of: A) directions for further improvement of model performance and efficiency; B) technical considerations for practical on-satellite deployment.

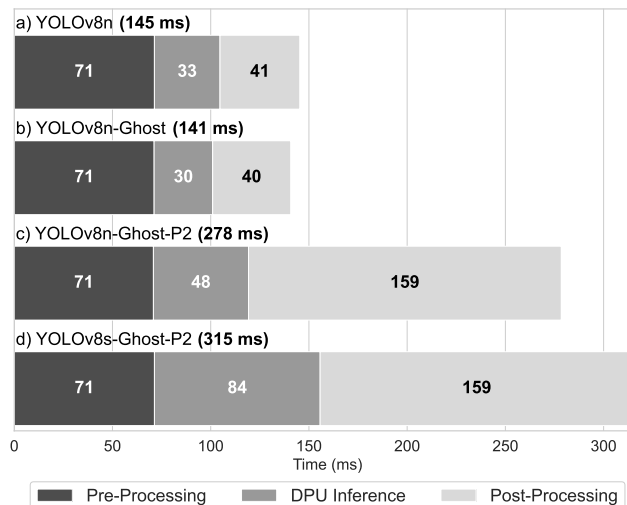


Figure 7: Processing time breakdown of YOLOv8 variants on Kria KV260 (average from 100 inference runs).

5.1 Potential further performance and efficiency improvement

The residual performance gap between our FPGA-deployed model and larger GPU-based models is mainly in near-shore detection (0.447 vs. 0.428–0.525 F1 score of the xView3-SAR top-5 entries) and fishing classification (0.769 vs. 0.797–0.834 F1 score), and is likely partly due to our focus on inference efficiency. All five of our benchmark models employ model ensembles, test-time augmentation (TTA), or both during inference, which improve performance but which we choose not to implement here due to increased computational cost. The performance difference may also reflect a fundamental limit of the simple single-stage detection architecture for this challenging task, as several winning xView3-SAR competitors used architectures such as UNet or HRNet, which are large but excel at preserving the fine-grained spatial details crucial for distinguishing small vessels from complex coastal clutter. Vision Transformers may be a viable lightweight alternative to CNNs for this task, and have shown promise both for SAR vessel detection [3, 26, 84], and for maintaining performance at low bit-width, e.g. binary and ternary compression [30, 77]. Alternatively, further performance improvements may be possible by combining our model with statistical approaches, such as CFAR [73], or through leveraging additional auxiliary information (e.g. coastline, static infrastructure or areas of interest) or historical imagery.

Despite the overall speed of our current analysis pipeline, there are also opportunities to further improve our model's speed of analysis and efficiency. There is a notable computational bottleneck in both pre- and post-processing operations, with these steps consuming 83% of the total inference time

(71 ms and 159 ms respectively, in comparison to just 48 ms for inference itself, see Fig. 7). The post-processing overhead is the largest, and arises from additional tens of thousands of outputs generated by the high-resolution P2 feature map, which require decoding into bounding box predictions. Simple modifications to our decoding code already achieved a 28× efficiency improvement over the baseline Vitis AI implementation, reducing processing time from 4400 ms to 159 ms, and we consider two additional approaches to reduce this further. We first test a hybrid strategy where the P2 model variant is only used on the more challenging near-shore areas of each scene (6% of the test dataset, identified using bathymetry, but could also be identified from SAR metadata and an onboard look-up table), while the baseline YOLOv8n model processes the remaining regions. This approach significantly reduces the average total inference time, from 278 ms to 153 ms per chip across the test set, but also leads to a significant performance drop of 4.1% and 4.3% in overall detection and fishing classification respectively compared to our preferred model. Alternatively, we also conducted preliminary experiments to offload the decoding step to the PL using Vitis HLS for operation parallelization. The synthesis results show a moderate latency reduction to 133 ms, and importantly, this approach also decreases PS–PL data transfer. This is a promising avenue for real-world deployment, running in parallel to the DPU by leveraging the unused resources of the FPGA's PL (see Table 1 for utilization). We leave deployment on hardware to future work. Pre-processing also likely carries unnecessary overhead due to ingestion of 32-bit floating point TIFF files and then conversion to INT8 images; this could be reduced by storing image chips in lower precision, and by implementing image normalization operations directly on the FPGA as per the post-processing.

Additional efficiency gains could come from systematic optimization of the DPU configuration, e.g. by varying parallelism levels, core counts, and operating frequencies; this is beyond current scope but 2× improvement in power efficiency has previously been demonstrated on identical hardware [27]. Finally, we could investigate quantizing our model to lower bit-widths (e.g. 4-bit [78] or 1-bit [82]), employing mixed-precision techniques [53, 56], pruning model parameters to reduce computational load and memory usage, or knowledge distillation to transfer knowledge from the quantized model into an even smaller and faster student model.

5.2 Towards practical on-satellite deployment

First, our vessel detection implementation would also need to be deployed alongside low-power and fast (<1 minute) FPGA SAR processing (as already demonstrated by [7, 10, 39]), which converts raw SAR data (L0) to the rasterized scenes

(L1) needed as input. In addition, the detection threshold would need to be adjusted to reflect specific mission requirements. Here we adopt a relatively conservative threshold, which produces the best results in evaluation using the xView3-SAR competition metrics, and results in fewer false positives (FP: 10,780 vs. 12,369) but more false negatives (FN: 15,670 vs. 13,345) in comparison to the winning model. However, where minimization of missed detections is critical, optimizing the threshold for high recall (lower false negative rate) would be more suitable, accepting the increased computational cost of processing additional candidate detections.

We also note that limited on-satellite storage may place constraints on the SAR processing and analysis pipeline, particularly for the bathymetry data used here as an auxiliary input data channel. These data are not large (global dataset 7.5 GB, 4 GB compressed [23]) in comparison to storage capacity of large satellites (Pléiades Neo has 1.75 TB), so could be stored on space-hardened storage, periodically updated from ground stations, and co-registered with observation areas using onboard logic. Alternatively, the bathymetry could be omitted at a minor cost to performance (reduction of classification and near-shore detection by 1.3% on average; see ablation results in Table 3.B), or could be replaced with alternative contextual information generated onboard from SAR data (e.g. land masks from thresholding using Otsu's method [37]), at the cost of additional onboard compute.

6 Summary

In this study we have demonstrated that highly-performant ML SAR vessel detection is possible in power-restricted on-satellite systems, with the quasi real-time latency (<1 min for a ~40,000 km² SAR scene) required for real-world maritime intelligence and satellite autonomy applications. We have achieved this through systematic exploration and combination of previous CNN model adaptations to create a tailored model for this task, and through deployment and testing of our new model on the Kria KV260 platform, operating within a CubeSat-relevant sub-10W power budget. In the long term, development of on-board ML has potential to transform satellite constellations from passive data collection systems into active monitoring networks capable of immediate response to events of interest; ultimately enhancing global security, environmental protection, and disaster response.

Acknowledgments

This work was supported by the Turing's Defence and Security programme through a partnership with Dstl. This research used the Baskerville Tier 2 HPC; funded by EPSRC & UKRI (EP/T022221/1, EP/W032244/1) and operated by Advanced Research Computing, U. Birmingham. Copernicus Sentinel-1 SAR data are provided courtesy of ESA.

References

- [1] GEBCO Bathymetric Compilation Group 2019. 2019. The GEBCO2019 Grid - a continuous terrain model of the global oceans and land. *Published Data Library (PDL)* (2019). doi:10.5285/836F016A-33BE-6DDC-E053-6C86ABC0788E
- [2] European Space Agency. 2020. MicroSAR Integrated Ground Segment.
- [3] Mahmoud Ahmed, Naser El-Sheimy, and Henry Leung. 2024. A Novel Detection Transformer Framework for Ship Detection in Synthetic Aperture Radar Imagery Using Advanced Feature Fusion and Polarimetric Techniques. *Remote Sensing* 16, 20 (2024). doi:10.3390/rs16203877
- [4] Carles Araguz, Elisenda Bou-Balust, and Eduard Alarcón. 2018. Applying autonomy to distributed satellite systems: Trends, challenges, and future prospects. *Systems Engineering* 21, 5 (2018), 401–416. doi:10.1002/sys.21428
- [5] Ch Muhammad Awais, Marco Reggiannini, Davide Moroni, and Emanuele Salerno. 2025. A Survey on SAR ship classification using Deep Learning. arXiv:2503.11906 [cs.CV]
- [6] Yifang Ban, Puzhao Zhang, Andrea Nascetti, Alexandre R. Bevington, and Michael A. Wulder. 2020. Near Real-Time Wildfire Progression Monitoring with Sentinel-1 SAR Time Series and Deep Learning. *Scientific Reports* 10, 1 (28 Jan 2020), 1322. doi:10.1038/s41598-019-56967-x
- [7] Emilio Isaac Baungarten-Leon, Gustavo Daniel Martín-del Campo-Becerra, Susana Ortega-Cisneros, Maron Schlemmon, Jorge Rivera, and Andreas Reigber. 2023. Towards On-Board SAR Processing with FPGA Accelerators and a PCIe Interface. *Electronics* 12, 12 (2023). doi:10.3390/electronics12122558
- [8] Piero Boccardo and Fabio Giulio Tonolo. 2015. Remote Sensing Role in Emergency Mapping for Disaster Response. In *Engineering Geology for Society and Territory - Volume 5*, Giorgio Lollino, Andrea Manconi, Fausto Guzzetti, Martin Culshaw, Peter Bobrowsky, and Fabio Luino (Eds.). Springer International Publishing, Cham, 17–24.
- [9] Alexey Bochkovskiy, Chien-Yao Wang, and Hong-Yuan Mark Liao. 2020. YOLOv4: Optimal Speed and Accuracy of Object Detection. arXiv:2004.10934 [cs.CV]
- [10] Helko Breit, Srikanth Mandapati, and Ulrich Balss. 2021. An FPGA/MPSoC based low latency onboard SAR processor. In *2021 IEEE International Geoscience and Remote Sensing Symposium IGARSS*. IEEE, 5159–5162.
- [11] Thanh Dang Bui and Tra My Do Le. 2025. Ghost-Attention-YOLOv8: Enhancing Rice Leaf Disease Detection with Lightweight Feature Extraction and Advanced Attention Mechanisms. *AgriEngineering* 7, 4 (2025). doi:10.3390/agriengineering7040093
- [12] Mingxiang Cao, Weiying Xie, Jie Lei, Jiaqing Zhang, Daixun Li, and Yunsong Li. 2024. Multi-scale direction-aware SAR object detection network via global information fusion. arXiv:2312.16943 [cs.CV]
- [13] Chenyi Chen, Ming-Yu Liu, Oncel Tuzel, and Jianxiong Xiao. 2016. R-CNN for small object detection. In *Asian conference on computer vision*. Springer, 214–230.
- [14] Jierun Chen, Shiu-hong Kao, Hao He, Weipeng Zhuo, Song Wen, Chul-Ho Lee, and S.-H. Gary Chan. 2023. Run, Don't Walk: Chasing Higher FLOPS for Faster Neural Networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 12021–12031.
- [15] Becky Cudzilo, K.C. Foley, and Chandler Smith. 2012. The Ability of a Small Satellite Constellation to Tip and Cue Other Commercial Assets. In *26th Annual AIAA/USU Conference on Small Satellites*.
- [16] Khalid El-Darymli, Christoph Gierull, and Katerina Biron. 2024. Deep learning vs. K-CFAR for ship detection in spaceborne SAR imagery. 8037–8040. doi:10.1109/IGARSS53475.2024.10640656
- [17] European Space Agency. 2012. *Sentinel-1: ESA's Radar Observatory Mission for GMES Operational Services*. Special Publications, Vol. 1322. ESA Communications, Noordwijk, The Netherlands. 86 pages. ESA SP, ISSN 0379-6566. Volume 1 of SENTINEL / production ed.: Karen Fletcher.
- [18] European Space Agency (ESA). 2011. European Data Relay Satellite System (EDRS). <https://eda.europa.eu/docs/documents/EDRS.pdf> Accessed on 7 October 2025.
- [19] Nan Fang, Liyuan Li, Xiaoxuan Zhou, Wencong Zhang, and Fansheng Chen. 2025. An FPGA-Based Hybrid Overlapping Acceleration Architecture for Small-Target Remote Sensing Detection. *Remote Sensing* 17, 3 (2025). doi:10.3390/rs17030494
- [20] Fei Feng, Yu Hu, Weipeng Li, and Feiyan Yang. 2024. Improved YOLOv8 algorithms for small object detection in aerial imagery. *Journal of King Saud University - Computer and Information Sciences* 36, 6 (2024), 102113. doi:10.1016/j.jksuci.2024.102113
- [21] Yang Gao, Wei Hua, Hongnan Tian, Zhiqiang Sun, Muhui Cai, and Yushan Guo. 2022. Object Detection for SAR Ship Images Based on YOLOv5 Deployed On Jetson AGX Xavier. In *2022 3rd China International SAR Symposium (CISS)*. 1–4. doi:10.1109/CISS57580.2022.9971242
- [22] Markus Gardill, Witold Kinsner, Jan Budroweit, Akram Al-Hourani, Emily Dunkel, Jason Swope, David Evans, and Maximilian Staebler. 2023. Towards Space Edge Computing and Onboard AI for Real-Time Teleoperations. In *IEEE International Conference on Cognitive Informatics and Cognitive Computing*.
- [23] GEBCO Bathymetric Compilation Group 2025. 2025. The GEBCO_2025 Grid - a continuous terrain model for oceans and land at 15 arc-second intervals. doi:10.5285/37c52e96-24ea-67ce-e063-7086abc05f29
- [24] Gianluca Giuffrida, Luca Fanucci, Gabriele Meoni, Matej Batič, Léonie Buckley, Aubrey Dunne, Chris van Dijk, Marco Esposito, John Hefele, Nathan Vercruyssen, Gianluca Furano, Massimiliano Pastena, and Josef Aschbacher. 2022. The ϕ -Sat-1 Mission: The First On-Board Deep Neural Network Demonstrator for Satellite Earth Observation. *IEEE Transactions on Geoscience and Remote Sensing* 60 (2022), 1–14. doi:10.1109/TGRS.2021.3125567
- [25] Xiaomeng Guo and Baoyi Xu. 2024. SAR-NTV-YOLOv8: A Neural Network Aircraft Detection Method in SAR Images Based on Despeckling Preprocessing. *Remote Sensing* 16, 18 (2024). doi:10.3390/rs16183420
- [26] Ritwik Gupta, Shufan Li, Tyler Zhu, Jitendra Malik, Trevor Darrell, and Karttikeya Mangalam. 2024. xT: nested tokenization for larger context in large images. In *Proceedings of the 41st International Conference on Machine Learning (Vienna, Austria) (ICML '24)*. JMLR.org, Article 678, 12 pages.
- [27] Fumio Hamanaka, Takashi Odan, Kenji Kise, and Thiem Van Chu. 2023. An Exploration of State-of-the-Art Automation Frameworks for FPGA-Based DNN Acceleration. *IEEE Access* 11 (2023), 5701–5713. doi:10.1109/ACCESS.2023.3236974
- [28] Kai Han, Yunhe Wang, Qi Tian, Jianyuan Guo, Chunjing Xu, and Chang Xu. 2020. GhostNet: More Features from Cheap Operations. arXiv:1911.11907 [cs.CV]
- [29] Jian He and Chaobing Huang. 2024. YOLOv8-SARship: An Efficient and Accurate Detection Method for SAR Ship Images. In *5th International Conference on Computer Vision, Image and Deep Learning (CVIDL)*. 280–284. doi:10.1109/CVIDL62147.2024.10604104
- [30] Yefei He, Zhenyu Lou, Luoming Zhang, Jing Liu, Weijia Wu, Hong Zhou, and Bohan Zhuang. 2023. BiViT: Extremely Compressed Binary Vision Transformers. In *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*. 5628–5640. doi:10.1109/ICCV51070.2023.00520
- [31] Hank Heidt, Jordi Puig-Suari, Augustus S. Moore, Shinichi Nakasuka, and Robert J. Twiggs. 2000. CubeSat: A New Generation of Picosatellite for Education and Industry Low-Cost Space Experimentation.

- [32] Qibin Hou, Daquan Zhou, and Jiashi Feng. 2021. Coordinate Attention for Efficient Mobile Network Design. In *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 13708–13717. doi:10.1109/CVPR46437.2021.01350
- [33] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, Quoc V. Le, and Hartwig Adam. 2019. Searching for MobileNetV3. arXiv:1905.02244 [cs.CV]
- [34] Xueqin Huang, Ke Xu, Ji Chen, Ao Wang, Shengyao Chen, and Hongtao Li. 2024. Real-Time Processing of Ship Detection with SAR Image Based on FPGA. In *IGARSS 2024 - 2024 IEEE International Geoscience and Remote Sensing Symposium*. 8954–8957. doi:10.1109/IGARSS53475.2024.10640725
- [35] Sambhav Jain, Albert Gural, Michael Wu, and Chris Dick. 2020. Trained Quantization Thresholds for Accurate and Efficient Fixed-Point Inference of Deep Neural Networks. In *Proceedings of Machine Learning and Systems*, I. Dhillon, D. Papailiopoulos, and V. Sze (Eds.), Vol. 2. 112–128.
- [36] JAXA. n.d.. Advanced Land Observing Satellite-4 “Daichi-4” [Technical overview]. Retrieved from <https://global.jaxa.jp/projects/sat/alos4/>.
- [37] Kefeng Ji, Xiangguang Leng, Qingju Fan, Shilin Zhou, and Huanxin Zou. 2016. An land masking algorithm for ship detection in SAR images. In *2016 IEEE International Geoscience and Remote Sensing Symposium (IGARSS)*. 925–928. doi:10.1109/IGARSS.2016.7729234
- [38] Ming Kang, Chee-Ming Ting, Fung Fung Ting, and Raphaël C.-W. Phan. 2024. BGF-YOLO: Enhanced YOLOv8 with Multiscale Attentional Feature Fusion for Brain Tumor Detection. In *Proceedings of Medical Image Computing and Computer Assisted Intervention – MICCAI 2024*, Vol. LNCS 15008. Springer Nature Switzerland.
- [39] Murray Kerr, Stefania Tonetti, Stefania Cornara, Juan-Ignacio Bravo, Robert Hinz, Antonio Latorre, Francisco Membibre, Chiara Solimini, Stefan Wiehle, Helko Breit, Björn Tings, Otto Koudelka, Franz Teschl, Enrico Magli, Tiziano Bianchi, Andrea Migliorati, Paolo Motto Ros, Michele Caon, Riccardo Freddi, Michele Benetti, Fabio Milani, Guido Curci, Silvia Fraile, Lucia Garcia, Cecilia Marcos, and Aniello Fiengo. 2020. EO-ALERT: A Novel Architecture for the Next Generation of Earth Observation Satellites Supporting Rapid Civil Alert. In *71st International Astronautical Congress (IAC 2020)*.
- [40] Boshra Khalili and Andrew W. Smyth. 2024. SOD-YOLOv8—Enhancing YOLOv8 for Small Object Detection in Aerial Imagery and Traffic Scenes. *Sensors* 24, 19 (2024). doi:10.3390/s24196209
- [41] Andrey A. Kurekin, Benjamin R. Loveday, Oliver Clements, Graham D. Quartly, Peter I. Miller, George Wiafe, and Kwame Adu Agyekum. 2019. Operational Monitoring of Illegal Fishing in Ghana through Exploitation of Satellite Earth Observation and AIS Data. *Remote Sensing* 11, 3 (2019). doi:10.3390/rs11030293
- [42] Haitao Lang, Ruifu Wang, Shaoying Zheng, Siwen Wu, and Jialu Li. 2022. Ship Classification in SAR Imagery by Shallow CNN Pre-Trained on Task-Specific Dataset with Feature Refinement. *Remote Sensing* 14, 23 (2022). doi:10.3390/rs14235986
- [43] Phang Kok Leong. 2025. Securing Malaysia’s maritime environment using space-derived services. *Contemporary Issues in Air and Space Power* 3, 1 (2025), bp42323989. doi:10.58930/bp42323989
- [44] Yonggang Li, Weigang Zhu, and Bakun Zhu. 2021. SAR Image Nearshore Ship Target Detection in Complex Environment. In *2021 IEEE 5th Advanced Information Technology, Electronic and Automation Control Conference (IAEAC)*. 1964–1968. doi:10.1109/IAEAC50856.2021.9391019
- [45] Tsung-Yi Lin, Piotr Dollár, Ross Girshick, Kaiming He, Bharath Hariharan, and Serge Belongie. 2017. Feature Pyramid Networks for Object Detection. In *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 936–944. doi:10.1109/CVPR.2017.106
- [46] Can Liu, Kaige Wang, Qing Li, Fazhan Zhao, Kun Zhao, and Hongtu Ma. 2024. Powerful-IoU: More straightforward and faster bounding box regression loss with a nonmonotonic focusing mechanism. *Neural Networks* 170 (2024), 276–284. doi:10.1016/j.neunet.2023.11.041
- [47] Shu Liu, Lu Qi, Haifang Qin, Jianping Shi, and Jiaya Jia. 2018. Path Aggregation Network for Instance Segmentation. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 8759–8768. doi:10.1109/CVPR.2018.00913
- [48] Yonghang Luo, Ming Li, Guihao Wen, Yunfei Tan, and Chaoshan Shi. 2024. SHIP-YOLO: A Lightweight Synthetic Aperture Radar Ship Detection Model Based on YOLOv8n Algorithm. *IEEE Access* 12 (2024), 37030–37041. doi:10.1109/ACCESS.2024.3373893
- [49] Ningning Ma, Xiangyu Zhang, Hai-Tao Zheng, and Jian Sun. 2018. ShuffleNet V2: Practical Guidelines for Efficient CNN Architecture Design. In *Proceedings of the European Conference on Computer Vision (ECCV)*.
- [50] Tian J. Ma and Robert J. Anderson. 2023. Remote Sensing Low Signal-to-Noise-Ratio Target Detection Enhancement. *Sensors* 23, 6 (2023). doi:10.3390/s23063314
- [51] Abhas Maskey and Mengu Cho. 2020. CubeSatNet: Ultralight Convolutional Neural Network designed for on-orbit binary image classification on a 1U CubeSat. *Engineering Applications of Artificial Intelligence* 96 (2020), 103952. doi:10.1016/j.engappai.2020.103952
- [52] Fanlong Meng, Xiangyang Qi, and Huaitao Fan. 2024. LSR-Det: A Lightweight Detector for Ship Detection in SAR Images Based on Oriented Bounding Box. *Remote Sensing* 16, 17 (2024). doi:10.3390/rs16173251
- [53] Alexander Montgomerie-Corcoran, Petros Toupas, Zhewen Yu, and Christos-Savvas Bouganis. 2023. SATAY: A Streaming Architecture Toolflow for Accelerating YOLO Models on FPGA Devices. In *2023 International Conference on Field Programmable Technology (ICFPT)*. 179–187. doi:10.1109/ICFPT59805.2023.00025
- [54] Alberto Moreira, Pau Prats-Iraola, Marwan Younis, Gerhard Krieger, Irena Hajnsek, and Konstantinos P Papathanassiou. 2013. A tutorial on synthetic aperture radar. *IEEE Geoscience and remote sensing magazine* 1, 1 (2013), 6–43.
- [55] NASA-ISRO SAR (NISAR). 2018. *Mission Science Users’ Handbook*. Technical Report. NASA Jet Propulsion Laboratory. 261 pages. L: NISAR.
- [56] Duy Thanh Nguyen, Hyun Kim, and Hyuk-Jae Lee. 2021. Layer-Specific Optimization for Mixed Data Flow With Mixed Precision in FPGA Design for CNN-Based Object Detectors. *IEEE Transactions on Circuits and Systems for Video Technology* 31, 6 (2021), 2450–2464. doi:10.1109/TCSVT.2020.3020569
- [57] OECD. 2019. *The Space Economy in Figures: How Space Contributes to the Global Economy*. OECD Publishing, Paris. doi:10.1787/c5996201-en
- [58] Daliang Ouyang, Su He, Guozhong Zhang, Mingzhu Luo, Huaiyong Guo, Jian Zhan, and Zhijie Huang. 2023. Efficient Multi-Scale Attention Module with Cross-Spatial Learning. In *ICASSP 2023 - 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 1–5. doi:10.1109/icassp49357.2023.10096516
- [59] Fernando Paolo, Tsu ting Tim Lin, Ritwik Gupta, Bryce Goodman, Nirav Patel, Daniel Kuster, David Kroodsma, and Jared Dunnmon. 2022. xView3-SAR: Detecting Dark Fishing Activity Using Synthetic Aperture Radar Imagery. arXiv:2206.00897 [cs.CV]
- [60] RangeKing. 2023. Ultralytics YOLOv8 Architecture Diagram. <https://github.com/ultralytics/ultralytics/issues/189>. Accessed: 2025-06-13.
- [61] Emilio Rapuano, Gabriele Meoni, Tommaso Pacini, Gianmarco Dinelli, Gianluca Furano, Gianluca Giuffrida, and Luca Fanucci. 2021. An FPGA-Based Hardware Accelerator for CNNs Inference on Board Satellites: Benchmarking with Myriad 2-Based Solution for the CloudScout Case Study. *Remote Sensing* 13, 8 (2021). doi:10.3390/rs13081518

- [62] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. 2016. You Only Look Once: Unified, Real-Time Object Detection. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE Computer Society, Los Alamitos, CA, USA, 779–788. doi:10.1109/CVPR.2016.91
- [63] Alfredo Renga, Maria Graziano, Marco D'Errico, A. Moccia, and A. Cecchini. 2011. SAR-based sea traffic monitoring: a reliable approach for Maritime Surveillance. *Proceedings of SPIE - The International Society for Optical Engineering* 8179. doi:10.1117/12.897986
- [64] Vít Růžička, Gonzalo Mateo-García, Chris Bridges, Chris Brunskill, Cormac Purcell, Nicolas Longépé, and Andrew Markham. 2023. Fast Model Inference and Training On-Board of Satellites. In *IGARSS 2023 - 2023 IEEE International Geoscience and Remote Sensing Symposium*. 2002–2005. doi:10.1109/IGARSS52108.2023.10282715
- [65] Rehan Mehmood Sabir, Kashif Mehmood, Abid Sarwar, Muhammad Safdar, Nalain E. Muhammad, Nimra Gul, Fareeha Athar, Muhammad Danish Majeed, Jaffar Sattar, Zafran Khan, Mariam Fatima, Muhammad Adnan Hussain, and Hafiz Muhammad Bilawal Akram. 2024. *Remote Sensing and Precision Agriculture: A Sustainable Future*. Springer Nature Switzerland, 75–103. doi:10.1007/978-3-031-63430-7_4
- [66] Mingxing Tan and Quoc Le. 2019. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. In *Proceedings of the 36th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 97)*, Kamalika Chaudhuri and Ruslan Salakhutdinov (Eds.). PMLR, 6105–6114.
- [67] Hashir Tanveer, Nicholas R. Magliocca, and Kevin M. Curtin. 2025. Identifying narco-trafficking landing zones using satellite imagery and geospatial indicators in Costa Rica. *GeoJournal* 90, 1 (23 Jan 2025), 31. doi:10.1007/s10708-024-11279-0
- [68] Bill Tao, Maleeha Masood, Indranil Gupta, and Deepak Vasisht. 2023. Transmitting, fast and slow: Scheduling satellite traffic through space and time. In *Proceedings of the 29th Annual International Conference on Mobile Computing and Networking*. 1–15.
- [69] Kathiravan Thangavel, Roberto Sabatini, Alessandro Gardi, Kavindu Ranasinghe, Samuel Hilton, Pablo Servidia, and Dario Spiller. 2024. Artificial Intelligence for Trusted Autonomous Satellite Operations. *Progress in Aerospace Sciences* 144 (2024), 100960. doi:10.1016/j.paerosci.2023.100960
- [70] Deepak Vasisht, Jayanth Shenoy, and Ranveer Chandra. 2021. L2D2: Low latency distributed downlink for LEO satellites. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*. 151–164.
- [71] Yuanyuan Wang, Chao Wang, Hong Zhang, Yingbo Dong, and Sisi Wei. 2019. A SAR Dataset of Ship Detection for Deep Learning under Complex Backgrounds. *Remote Sensing* 11, 7 (2019). doi:10.3390/rs11070765
- [72] Shunjun Wei, Xiangfeng Zeng, Qizhe Qu, Mou Wang, Hao Su, and Jun Shi. 2020. HRSID: A High-Resolution SAR Images Dataset for Ship Detection and Instance Segmentation. *IEEE Access* 8 (2020), 120234–120254. doi:10.1109/ACCESS.2020.3005861
- [73] Xue Wen, Shaoming Zhang, Jianmei Wang, Tangjun Yao, and Yan Tang. 2024. A CFAR-Enhanced Ship Detector for SAR Images Based on YOLOv5s. *Remote Sensing* 16, 5 (2024). doi:10.3390/rs16050733
- [74] Sanghyun Woo, Jongchan Park, Joon-Young Lee, and In So Kweon. 2018. CBAM: Convolutional Block Attention Module. In *Proceedings of the European Conference on Computer Vision (ECCV)*.
- [75] Xin Xiao and Xinlong Feng. 2023. Multi-Object Pedestrian Tracking Using Improved YOLOv8 and OC-SORT. *Sensors* 23 (10 2023), 8439. doi:10.3390/s23208439
- [76] Qizhe Xie, Minh-Thang Luong, Eduard Hovy, and Quoc V Le. 2020. Self-training with noisy student improves imagenet classification. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 10687–10698.
- [77] Sheng Xu, Yanjing Li, Teli Ma, Bo-Wen Zeng, Baochang Zhang, Peng Gao, and Jinhu Lv. 2022. TerViT: An Efficient Ternary Vision Transformer. *ArXiv abs/2201.08050* (2022).
- [78] Zhihong Yan, Bingqian Zhang, and Dong Wang. 2024. An FPGA-Based YOLOv5 Accelerator for Real-Time Industrial Vision Applications. *Micromachines* 15, 9 (2024). doi:10.3390/mi15091164
- [79] Geng Yang, Jie Lei, Weiying Xie, Zhenman Fang, Yunsong Li, Jiaxuan Wang, and Xin Zhang. 2022. Algorithm/Hardware Codesign for Real-Time On-Satellite CNN-Based Ship Detection in SAR Imagery. *IEEE Transactions on Geoscience and Remote Sensing* 60 (2022), 1–18. doi:10.1109/TGRS.2022.3161499
- [80] Muhammad Yasir, Shanwei Liu, Saied Pirasteh, Mingming Xu, Hui Sheng, Jianhua Wan, Felipe A.P. de Figueiredo, Fernando J. Aguilar, and Jonathan Li. 2024. YOLOShipTracker: Tracking ships in SAR images using lightweight YOLOv8. *International Journal of Applied Earth Observation and Geoinformation* 134 (2024), 104137. doi:10.1016/j.jag.2024.104137
- [81] Muhammad Yasir, Abdoul Jelil Niang, Md Sakaouth Hossain, Qamar Ul Islam, Qian Yang, and Yuhang Yin. 2023. Ranking Ship Detection Methods Using SAR Images Based on Machine Learning and Artificial Intelligence. *Journal of Marine Science and Engineering* 11, 10 (2023). doi:10.3390/jmse11101916
- [82] Dezheng Zhang, Zehan Liang, Rui Cen, Zhihong Yan, Rui Wan, and Dong Wang. 2025. HE-BiDet: A Hardware Efficient Binary Neural Network Accelerator for Object Detection in SAR Images. *Micromachines* 16, 5 (2025). doi:10.3390/mi16050549
- [83] Tianwen Zhang, Xiaoling Zhang, Jianwei Li, Xiaowo Xu, Baoyou Wang, Xu Zhan, Yanqin Xu, Xiao Ke, Tianjiao Zeng, Hao Su, Israr Ahmad, Dece Pan, Chang Liu, Yue Zhou, Jun Shi, and Shunjun Wei. 2021. SAR Ship Detection Dataset (SSDD): Official Release and Comprehensive Data Analysis. *Remote Sensing* 13, 18 (2021). doi:10.3390/rs13183690
- [84] Yang Zhang, Jingyi Cao, Yanan You, and Yuanyuan Qiao. 2025. DenSe-AdViT: A novel Vision Transformer for Dense SAR Object Detection. *arXiv:2504.13638 [cs.CV]*
- [85] Shuguang Zhao, Ronghao Tao, and Fengde Jia. 2024. DML-YOLOv8-SAR image object detection algorithm. *Signal, Image and Video Processing* 18, 10 (2024), 6911–6923. doi:10.1007/s11760-024-03361-4
- [86] Yaoyao Zhong, Weihong Deng, Mei Wang, Jiani Hu, Jianteng Peng, Xunqiang Tao, and Yaohai Huang. 2019. Unequal-training for deep face recognition with long-tailed noisy data. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 7812–7821.
- [87] Shiliang Zhu and Min Miao. 2024. Lightweight high-precision SAR ship detection method based on YOLOv7-LDS. *PLOS ONE* 19, 2 (02 2024), 1–26. doi:10.1371/journal.pone.0296992
- [88] Xiao Xiang Zhu, Sina Montazeri, Mohsin Ali, Yuansheng Hua, Yuanyuan Wang, Lichao Mou, Yilei Shi, Feng Xu, and Richard Bamler. 2021. Deep Learning Meets SAR: Concepts, models, pitfalls, and perspectives. *IEEE Geoscience and Remote Sensing Magazine* 9, 4 (2021), 143–172. doi:10.1109/MGRS.2020.3046356

Received 20 June 2025; revised 8 October 2025; accepted 19 October 2025